Foundational Verification of Probabilistic Programs

Mechanising the Lilac probabilistic separation logic in Lean

Edwin Fernando

June 18, 2026

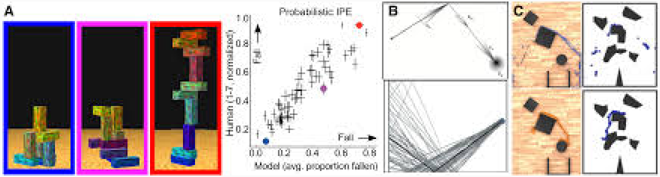
Department of Computing, Imperial College London

Probabilistic Programming is Everywhere

Probabilistic programming is everywhere

The most general feature of a PPL — sampling from a distribution — lives in the standard library of every major language.

- **Cryptography** — protocols, ZK proofs
- **Differential privacy**
- **Randomised algorithms**
- **Distributed systems**
- **Bayesian inference** & probabilistic ML
- **Scientific simulation** — climate, physics
- **Robotics & perception** — intuitive physics



Discrete vs. continuous

The dividing line that matters runs through the applications.

Discrete	Continuous
Coin flips, Bernoulli / categorical choices, dice. Cryptography, differential privacy, randomised algorithms — traditional CS, well served by *existing* logics.	Gaussian noise, sensor models, Bayesian priors over $\mathbb{R}$, physical quantities. Scientific simulation, Bayesian inference, ML — where the real-world modelling happens.

Continuous distributions are where prior verification work fell short — and where this project makes its **first mechanised** contribution.

Verifying probabilistic programs is notoriously hard

- Conventional testing fails: a rare *correct* outlier execution and a genuine bug look *identical*.
- Correctness is a property of the whole distribution, not of any single run.
- Reasoning about independence and conditioning is subtle and error-prone by hand.

Verifying probabilistic programs is notoriously hard

- Conventional testing fails: a rare *correct* outlier execution and a genuine bug look *identical*.
- Correctness is a property of the whole distribution, not of any single run.
- Reasoning about independence and conditioning is subtle and error-prone by hand.

So we need proof

Machine-checked, **foundational** arguments about distributions — resting on nothing but the axioms of mathematics.

Contribution

The first mechanisation of foundational verification

for probabilistic programs

supporting continuous distributions.

Contribution

1. Soundness proofs
2. Interactivity using Iris

The first mechanisation of foundational verification

for probabilistic programs

supporting continuous distributions.

Contribution

using a theorem prover like Lean
or Rocq
Highest achievable assurance
Assuming only the foundational
axioms of mathematics

1. Soundness proofs
2. Interactivity using Iris

The first mechanisation of foundational verification

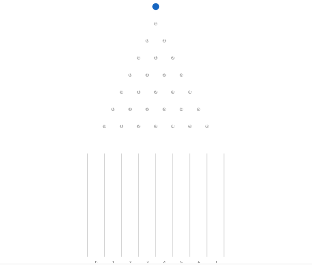
for probabilistic programs

supporting continuous distributions.

A program is a distribution over runs

$$\text{flip} : (m : \mathbb{R}) \to \text{Measure}\ \{m, m+1\}$$

an individual “run” of the program



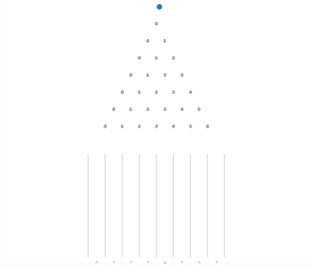
Honing in on a single marble

$$X_0 \leftarrow \text{flip}\ 0$$
$$X_1 \leftarrow \text{flip}\ X_0$$
$$X_2 \leftarrow \text{flip}\ X_1$$
$$\text{ret}\ X_2$$

$$|||$$

$$\text{for}\ (3,\ \text{flip}\ 0,\ \lambda X.\ \text{flip}\ X)$$

The program as a whole



All the marbles at once

Contributions, “revised”

Now that the motivation is in place: what did we mechanise, and why does it matter?

- Lilac is a probabilistic separation logic whose guiding idea is

  $$\text{“}\textit{separation} = \textit{independence of random variables}\text{”}$$

  The separating conjunction $P * Q$ asserts that the random variables described by P and Q are probabilistically independent — and Lilac supports **continuous** distributions.
- We give the **first mechanisation of Core[1] Lilac in Lean 4**, the first mechanisation of *any* probabilistic separation logic in Lean.

[1]Extended Lilac adds reasoning about conditioning

What was mechanised

1. **Soundness, from the ground up.** Core Lilac is built as a model over probability spaces with finite footprint, and every program-logic proof rule is proved sound against that model:

`wp_ret` `wp_bind` `wp_unif` `wp_cons` `wp_frame`

What was mechanised

1. **Soundness, from the ground up.** Core Lilac is built as a model over probability spaces with finite footprint, and every program-logic proof rule is proved sound against that model:

   `wp_ret`   `wp_bind`   `wp_unif`   `wp_cons`   `wp_frame`

2. **An interactive proof interface.** Lilac is instantiated as a bunched-implication logic on the Lean port of Iris, so proofs are carried out in the *Iris proof mode*.

What was mechanised

1. **Soundness, from the ground up.** Core Lilac is built as a model over probability spaces with finite footprint, and every program-logic proof rule is proved sound against that model:

   `wp_ret`   `wp_bind`   `wp_unif`   `wp_cons`   `wp_frame`

2. **An interactive proof interface.** Lilac is instantiated as a bunched-implication logic on the Lean port of Iris, so proofs are carried out in the *Iris proof mode*.
3. **Worked examples.** Specifications of sampling programs proved in that proof mode — including constructing a separating conjunct of two independent uniform samples.

Separation *is* independence — formally

The separating conjunction splits a probability space into an independent product $\sigma_1 \star \sigma_2$:

```
sep P Q := ⟨fun σ ↦ ∃ σ₁ σ₂, ∃ (σ₁₂ : ✓'(σ₁ ⋆ σ₂)),
                          ↓σ₁₂ ≤ σ ∧ P.1 σ₁ ∧ Q.1 σ₂, …⟩
```

Separation *is* independence — formally

The separating conjunction splits a probability space into an independent product $\sigma_1 \star \sigma_2$:

```
sep P Q := ⟨fun σ ↦ ∃ σ₁ σ₂, ∃ (σ₁₂ : ✓'(σ₁ ⋆ σ₂)),
                         ↓σ₁₂ ≤ σ ∧ P.1 σ₁ ∧ Q.1 σ₂, …⟩
```

This is not just a slogan — it is a theorem. Owning two random variables separately entails that they are genuinely independent:

```
lemma indep_of_own {X Y : RV ⟪Ty.real⟫} (Ω : PSp)
    (ownXY : (iprop(own X ∗ own Y) : LProp).1 Ω) :
    IndepFun (μ := Ω.μ) X Y
```

Separation *is* independence — formally

The separating conjunction splits a probability space into an independent product $\sigma_1 \star \sigma_2$:

```
sep P Q := ⟨fun σ ↦ ∃ σ₁ σ₂, ∃ (σ₁₂ : ✓'(σ₁ ⋆ σ₂)),
                          ↓σ₁₂ ≤ σ ∧ P.1 σ₁ ∧ Q.1 σ₂, …⟩
```

This is not just a slogan — it is a theorem. Owning two random variables separately entails that they are genuinely independent:

```
lemma indep_of_own {X Y : RV ⟪Ty.real⟫} (Ω : PSp)
    (ownXY : (iprop(own X ∗ own Y) : LProp).1 Ω) :
    IndepFun (μ := Ω.μ) X Y
```

$$(\text{own } X * \text{own } Y) \implies X \perp\!\!\!\perp Y$$

with the magic wand $-\!\!*$ available for hypothetical reasoning.